

LC 1928: Minimum Cost to Reach Destination in Time

Algorithm Notes & Implementation (Meta)

Authored by: Arpit Raj, LNMIIT Jaipur | July 26, 2026

Concept & Intuition

- Each edge contains `next node` , `traveltime` , and every node contains a `passing fee[node]` .
- We maintain a state `dist[node][time]` which represents the minimum cost to reach this node using exactly this much time.
- A priority queue stores `{cost, {node, timeused}}` . The priority is dictated by cost because that is what we are minimizing.

State Transitions

- For every neighbor, we calculate `newtime = currtime + edgetime` .
- If `newtime <= maxtime` , then `newcost = currcost + passingfee[nextnode]` .
- If `(newcost < dist[nextnode][newtime])` , then we update the distance and push it to the queue.

States: $N \times \text{maxtime}$

Each state relaxes all edges.

TC: $O(\text{maxtime} \times E \log(N \times \text{maxtime}))$

C++ Solution

```
class Solution {
public:
    int minCost(int maxTime, vector<vector<int>>& edges, vector<int>&
    passingFees) {
        int n = passingFees.size();
        vector<vector<pair<int, int>>> adj(n);

        for(auto& it : edges){
            int u = it[0];
            int v = it[1];
            int w = it[2];

            adj[u].push_back({v, w});
            adj[v].push_back({u, w});
        }

        // We need dist[node][time] representing cost to reach node in given
        time
        // We need priority queue to store {cost, {node, time}};
        priority_queue<
            pair<int, pair<int, int>>,
            vector<pair<int, pair<int, int>>>,
            greater<pair<int, pair<int, int>>>
        > pq;

        vector<vector<int>> dist(n, vector<int>(maxTime + 1, INT_MAX));

        dist[0][0] = passingFees[0];
        pq.push({passingFees[0], {0, 0}});

        while(!pq.empty()){
            int currnode = pq.top().second.first;
            int currtime = pq.top().second.second;
            int currcost = pq.top().first;

            pq.pop();

            if(currcost > dist[currnode][currtime]) continue;
            if(currnode == n - 1) return currcost;

            for(auto &it : adj[currnode]){
                int newnode = it.first;
                int addt = it.second;

                int newtime = currtime + addt;
                if(newtime > maxTime) continue;

                int newcost = currcost + passingFees[newnode];
```

```
        if(newcost < dist[newnode][newtime]){
            pq.push({newcost, {newnode, newtime}});
            dist[newnode][newtime] = newcost;
        }
    }
}
return -1;
}
};
```